



PDF Download
3705013.pdf
16 January 2026
Total Citations: 1
Total Downloads: 491

 Latest updates: <https://dl.acm.org/doi/10.1145/3705013>

RESEARCH-ARTICLE

A Comprehensive Model of Automated Evaluation of Difficulty in Platformer Games

YANNICK FRANCILLETTE, University of Quebec at Chicoutimi,
Chicoutimi-Jonquiere, QC, Canada

HUGO TREMBLAY, University of Quebec at Chicoutimi, Chicoutimi-
Jonquiere, QC, Canada

BRUNO BOUCHARD, University of Quebec at Chicoutimi, Chicoutimi-
Jonquiere, QC, Canada

BOB ANTOINE JERRY MÉNÉLAS, University of Quebec at Chicoutimi,
Chicoutimi-Jonquiere, QC, Canada

Open Access Support provided by:
University of Quebec at Chicoutimi

Published: 24 January 2025
Online AM: 16 December 2024
Accepted: 29 July 2024
Revised: 14 July 2024
Received: 22 December 2023

[Citation in BibTeX format](#)

A Comprehensive Model of Automated Evaluation of Difficulty in Platformer Games

YANNICK FRANCILLETTE, Informatique et Mathématique, Université du Québec à Chicoutimi, Chicoutimi, Canada

HUGO TREMBLAY, Informatique et Mathématique, Université du Québec à Chicoutimi, Chicoutimi, Canada

BRUNO BOUCHARD, Informatique et Mathématique, Université du Québec à Chicoutimi, Chicoutimi, Canada

BOB-ANTOINE MENELAS, Informatique et Mathématique, Université du Québec à Chicoutimi, Chicoutimi, Canada

Difficulty constitutes a key component of games. It both serves as motivation to play the game and as a way to control progression. Usually, a video game offers a gradual progression in gameplay difficulty, starting with easy levels that allow players to grasp the basic mechanics and incrementally introducing more challenging obstacles as they progress. Needless to say, the success of a game heavily relies on its ability to provide a well-balanced difficulty curve and a satisfying progression. Therefore, designing a game entails a complex and time-consuming process that involves extensive playtesting. One potential approach to address this challenge is the utilization of software tools capable of automatically evaluating the difficulty of game levels. In this article, we present a comprehensive model for automatically evaluating the difficulty levels of platformer games. Our model is based on the formal calculation of static danger zones within levels and the analysis of enemy movement patterns using simulated pheromones. To validate our model, we implemented it and conducted tests using the complete set of levels from the game *Super Mario Bros*. Furthermore, the article includes a detailed presentation of the model, the tools developed, a comparative analysis showing the computed results, and a discussion on the limitations and advantages.

CCS Concepts: • **Applied computing** → *Media arts*; • **Computing methodologies** → **Modeling methodologies**; • **Human-centered computing** → **Visualization techniques**;

Additional Key Words and Phrases: Video game, automatic difficulty evaluation, platformer games, level design, static difficulty, dynamic difficulty, executive (motor) difficulty, experiment and comparative results

Y. Francillette, H. Tremblay, B. Bouchard, and B.-A. Menelas contributed equally to this research.

Authors' Contact Information: Yannick Francillette, Informatique et Mathématique, Université du Québec à Chicoutimi, Chicoutimi, Quebec, Canada; e-mail: yannick_francillette@uqac.ca; Hugo Tremblay, Informatique et Mathématique, Université du Québec à Chicoutimi, Chicoutimi, Quebec, Canada; e-mail: hugo_tremblay2@uqac.ca; Bruno Bouchard, Informatique et Mathématique, Université du Québec à Chicoutimi, Chicoutimi, Quebec, Canada; e-mail: bruno.bouchard@uqac.ca; Bob-Antoine Menelas, Informatique et Mathématique, Université du Québec à Chicoutimi, Chicoutimi, Quebec, Canada; e-mail: bamenela@uqac.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2832-5516/2025/01-ART3

<https://doi.org/10.1145/3705013>

ACM Reference Format:

Yannick Francillette, Hugo Tremblay, Bruno Bouchard, and Bob-Antoine Menelas. 2025. A Comprehensive Model of Automated Evaluation of Difficulty in Platformer Games. *ACM Games* 3, 1, Article 3 (January 2025), 28 pages. <https://doi.org/10.1145/3705013>

1 Introduction

Video games have significantly evolved since their beginning and transformed from simple forms of entertainment to rich interactive experiences engaging players on multiple levels. Among the myriad elements that contribute to the intricacy of modern video games, the concept of “difficulty” stands as a fundamental cornerstone [26], which is often considered as the core component of player engagement and satisfaction. It encompasses a multifaceted imbrication of factors that not only challenge players but also shape their entire gaming experience. Additionally, the notion of difficulty relies on a complex and nuanced construct that transcends the simple measure of challenges faced by players. It can then be seen as the catalyst for challenge in a game and constitutes a key aspect of the design process that must be given serious thought by the game developers to be able to achieve their desired experience within their game [28]. Therefore, it is a time-consuming part of the design journey requiring the solicitation of many game testers. The viability of a game title greatly hinges on its capacity to deliver a carefully calibrated difficulty curve and a gratifying sense of advancement. As Chen [10] demonstrated using Csikszentmihalyi’s theory of flow, if the challenge is too simple, then the player get bored easily, but if it is too difficult, then they will feel anxious. Generally, the ideal state lies in the middle, where the player experiences a fulfilling zone.

In fact, the level of difficulty in a game experience is determined by the skill required of the player to progress. A skill can be defined as, “...a combination of mental and physical effort and capabilities... [Players] can vary heavily in skill, depending on their previous experiences, motor skills, cognitive capabilities in the game, etc.” [16].

Difficulty can serve many different purposes in a game, each of which has an impact on players and their overall experience [4]. It can also serve as the foundation or stepping stones for progression, helping players learn the game and keeping them engaged throughout their experience. This can be an obstacle that players must overcome to progress in a game, but it can serve as a punishment for a lack of mastery [28]. In addition, it also serves as a way to give sense of achievement to the player, to motivate them, re-enforcing is actual engagement in the game. Besides, it contributes to regulate the narrative pacing, to induce immersion, to prolong the gameplay, and to keep the player in the Flow zone [10]. It is, indeed, why game developers carefully consider and fine-tune difficulty level to strike the right balance and provide a satisfying, enjoyable, and memorable gaming adventure.

For further clarification, we can explain that the game industry basically employs two main strategies for designing difficulty. The first strategy is known as **Static Game Balance (SGB)**, which involves meticulously crafting and manually defining the difficulty of each game element by the design teams [23]. In SGB, one often adds static difficulty levels that can be selected directly by the user (easy, normal, hard, etc.). Significant examples include Nintendo *Mario Bros. series*,¹ the Firaxis hits *X-COM 2*,² or the popular *Minecraft*³ by Mojang Studios. SGB can also take the form of a single crafted difficulty setting in which the player can make, directly in the game,

¹[https://www.mariowiki.com/Super_Mario_\(series\)](https://www.mariowiki.com/Super_Mario_(series))

²<https://xcom.com/>

³<https://www.minecraft.net/>

different choices that will impact and calibrate its difficulty level. This way, the challenge will be adjusted by the player in an seamless way. For instance, in most **Role Playing Games (RPG)** [2], the player can choose to stay in one area longer to eliminate the enemies easily to improve the abilities of its avatar, thus reducing the challenge of the next dungeon. On the contrary, they could choose to stay for a shorter time, then progressing more quickly but with a weaker avatar and a more difficult challenge ahead. Another significant example is *Diablo II*⁴ from Blizzard Studios. The second strategy used by the industry is called **Dynamic Difficulty Adjustment (DDA)**, which refers to software or AI techniques that make a game adjust itself in real time to make the game better fit the player's abilities and therefore to improve the gaming experience [25]. These adjustments are implemented to maintain player engagement and interest. In the DDA approach, the system gathers various player data such as position, health, and ammo. After evaluating the player's conditions, the system selects the relevant data that reflect their level of immersion. By analyzing the player's flow state, the system then computes any necessary changes to the game. Finally, the game applies these modifications accordingly [36]. For instance, the changes can be decreasing or increasing the number of enemies. There are a lot of recent examples about the use of these techniques. For instance, the *Left 4 Dead*⁵ game by Valve uses an AI that constantly adapts the difficulty based on players' performance. If players are progressing easily, then the game introduces more powerful enemies to maintain the challenge. The *Halo*⁶ series also uses a form of DDA based on players' performance in-game. If they struggle, then the game provides more weapons and ammunition; meanwhile, if the players are excellent, then it ramps up the challenge.

In their study, Alexander et al. (2013) [1] examine the impact of both SGB and DDA systems on player enjoyment. They conducted tests with players of different skill levels who played their proprietary game. Both SGB and DDA offer unique advantages, drawbacks, and effects on player experience to different extents. For instance, DDA offers more adaptable and accommodating game-play experience that caters to individual player skill levels and preferences. It promotes inclusivity, engagement, and flow, ultimately enhancing the overall enjoyment and longevity of the game. However, SGB provides a consistent, predictable, and strategically focused experience for players who appreciate mastering challenges and working within a fixed set of rules. It motivates players to seek authenticity, long-term engagement, opportunity to develop, as well as refining their skills. Despite the ongoing debate surrounding the validity of one over the other in contemporary game design, both systems continue to exist and are used in multiple recent titles due to the specific roles they fulfill within the gaming industry[28].

The notion of difficulty in games involves two concepts that are important to distinguish. On the one hand, there is the intrinsic difficulty of the challenge proposed by the game. In their article, Denisova et al. define a challenge as a stimulating task or problem, pointing out that, in a game, one of the players' motivations is to respond to challenges. The term "difficulty" refers to the fact that something is hard to do [13]. Thus, a challenge poses a problem that players must be able to solve using their abilities to progress further in the game. Intrinsic difficulty therefore refers to the elements that are set in place to create a problem that is more or less difficult to solve. This includes, for example, the topology of a level, the number, type, and positioning of enemies, the setting of a timer, balancing, dynamic elements (such as mobile platforms, for example), and so on. Hence, in a level of a platform game like *Super Mario Bros.*, the more the level designer places elements that can kill the player, the more objectively difficult the challenge offered. The player is more likely to lose in this type of level than in another where there is only one hole to

⁴<https://diablo2.blizzard.com/fr-fr/>

⁵<https://www.l4d.com/>

⁶<https://www.halowaypoint.com/>

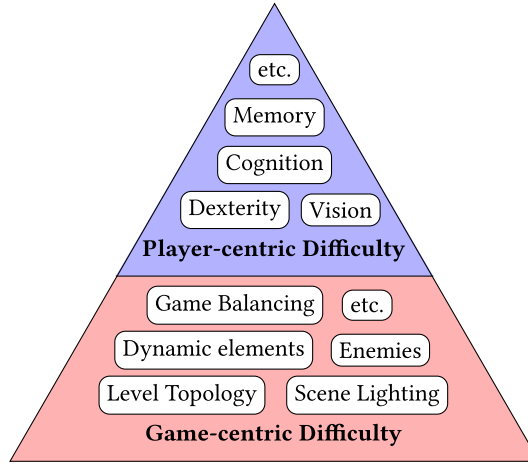


Fig. 1. Pyramid representing the two types of difficulty. The “etc.” label of game-centric difficulty refers to other elements of game-related difficulty such as image colorimetry, number and complexity of rules. In player-centric difficulty, the label “etc.” refers to other skills specific to the player such as motor skills and stamina.

avoid, with no timer and no enemy. We can call this intrinsic difficulty “**game-centric difficulty**.” On the other hand, there is extrinsic difficulty, which depends on the player’s skills. In fact, not all players have the same dexterity, analysis, or memorization skills. Therefore, the difficulty felt by players when faced with a given challenge will depend on the player’s abilities. For example, a player with better dexterity and motor coordination will give a lower estimate of the difficulty of a level with many holes and enemies than another player who is less advanced. This type of difficulty is extrinsic, since it depends on the player. We can call this extrinsic difficulty “**player-centric difficulty**.” Since player-centric difficulty is directly dependant on game-centric difficulty, the latter is the foundation upon which sits the player’s own experience. In other words, player-centric difficulty cannot exist without game-centric difficulty, which is itself a more fundamental or theoretical concept. Figure 1 shows a pyramid representation of these two types of difficulty. In this article, we focus on the analysis of “**game-centric difficulty**.”

According to Frampton [18], difficulty in games can be classified into three main categories. The first category is known as “*comprehensive*” difficulty, which refers to the level of challenge associated with fully understanding the core rules, objectives, and parameters of the game. For example, *Pac-Man*⁷ has straightforward rules, while games like *Pokémon*⁸ require players to memorize a vast amount of weaknesses, resistance types, and more, making them comprehensively difficult. The second type of difficulty is referred to as “*executive*” or “*motor*” difficulty. It encompasses physical abilities such as dexterity, precision, and speed in using game controllers. Platformers serve as examples in this category. The third and final category is known as “*strategic*” difficulty, which involves the mental/logical ability to make strategic/tactical choices in the game. Games such as *Starcraft 2*⁹ or *Civilization*¹⁰ are notable examples of strategic difficulty. Clearly, it is possible for a game to incorporate multiple types of difficulty simultaneously.

⁷<https://pacman.com/>

⁸<https://www.pokemon.com/>

⁹<https://starcraft2.blizzard.com/>

¹⁰<https://civilization.com/>

Finally, there are many game types or genres, each one proposing its own and unique form of difficulty [32]. For instance, Racing Games, such as *Dirt Rally 2*¹¹ or *Mario Kart*,¹² involve competing in races with various vehicles, aiming for fast times or outpacing opponents. In that kind of game, the difficulty is mainly executive and consists in piloting the vehicle, avoiding opponents' aggressive moves, and handling obstacles. In adventure games, such as *Fallout*¹³ or the cooperative game *A Way Out*,¹⁴ the system prioritizes storytelling, exploration, and puzzle-solving. Players engage in narrative-driven experiences set in immersive worlds. The difficulty is mainly the comprehensive type. In Simulation/Strategic games such as *Civilization VI* or *Master of Orion*,¹⁵ the challenge is more Strategic, while the players have to manage resources, build environments, and make strategic and tactical choices.

In summary, we can see that the design of game difficulty is particularly complex and depends of the type of the games, the chosen difficulty model. In the specific context of an SGB model, it involves a complex and time-consuming design process that requires extensive playtesting to achieve a well-balanced state. Some authors have proposed the utilization of **Machine Learning (ML)** techniques to assist game designers [29]. However, these approaches necessitate a huge amount of data to accurately learn the appropriate difficulty parameters, which may be limited during the design process with only a small number of playtesters. ML techniques excel in adjusting games that are already deployed and evolving live, like *World of Warcraft*,¹⁶ or in generating procedural content based on learned patterns [21]. Although, during the early design phase of a game, when the designer frequently modifies mechanics, tests various elements, and adjusts different parameters that directly impact the player's experience and difficulty levels, such ML techniques are not well suited.

In this article, we propose a novel formal model that aims to assist designers in creating platformer-type games. Our model is purely game-centric and enables the automatic measurement of executive or motor difficulty within a level by analyzing its interactive elements (the ones that produce collisions). It automatically identifies areas of potential (static) danger for players (holes, pikes, etc.) and simulates enemy movements (dynamic dangers), using a pheromone algorithm [17], to assess the motor skills difficulty of a specific level, eliminating the need for playtesters. We present the model, its implementation, and the results of a complete experiment conducted on all the levels of *Super Mario Bros*. The promising results demonstrate that our automatic algorithmic assessment model provides highly coherent outcomes that align with the original intentions of the *Mario's* designer in terms of difficulty and progression, which means that it follows the Nishikado difficulty progression model [20] (see Figure 8) implemented in *Super Mario Bros*. In addition, we provide a comparative analysis of our results with a study that we conducted directly with playtesters.

2 Related Works

Over the years, many works have been conducted to assess executive or motor difficulty in games [1, 3, 4, 24, 29, 30]. To be precise, executive or motor difficulty refers specifically to the level of spatial and temporal precision required by players to perform actions. This type of difficulty was a central focus in early video games such as *Pong*¹⁷ and *Space Invaders*.¹⁸ The simplest representation

¹¹<https://dirtrally2.dirtgame.com/>

¹²<https://mariokart8.nintendo.com/>

¹³<https://fallout.bethesda.net/en/games/fallout-4>

¹⁴<https://www.ea.com/games/a-way-out?setLocale=en-us>

¹⁵<https://masteroforion.eu/intro>

¹⁶<https://worldofwarcraft.blizzard.com/>

¹⁷<https://www.atari.com/>

¹⁸<https://www.taito.co.jp/en/>

of this difficulty involves avoiding static obstacles such as bottomless pits and spikes, commonly seen in games such as *Super Mario Bros.*,¹⁹ *Sonic the Hedgehog*,²⁰ and *Super Meat Boy*.²¹ Motor difficulty can vary, demanding either precise execution or quick succession of actions. For example, first-person shooter games like *Call of Duty*²² require accuracy during gunfights, while beat-em-up games like *Batman Arkham*²³ reward players for executing rapid button combinations. These games utilize motor difficulty principles in distinct ways, resulting in unique combat styles within their respective genres.

In our research context, we focus on the challenge of automatically measuring difficulty in video games, starting in this article with the executive difficulty. A handful of works [5, 12, 27, 31, 37] has been conducted over the years to propose avenues of solutions to that challenge. For instance, Aponte et al. [5] proposed to use a synthetic player to explore the parameter space of single player games for evaluating the level of difficulty. To achieve that, they developed a version of *Pac-Man* with only one ghost, which use a simple A* pathfinding algorithm. They also added a synthetic player with a partial view of the game comprising five parameters: The first one had four values, giving the direction of the nearest pellet; the four other dimensions describe the four *Pac-Man* directions. For each direction, *Pac-Man* knows whether it is facing a wall (0), a ghost one (1) or two (2) steps away from it, a free-of-ghosts path less than 18 steps long (3), or free-of-ghosts path longer than 18 steps long (4). The synthetic player AI was programmed with a **Markov Decision Process (MDP)**, using reinforcement learning with Q-Learning algorithm [22]. They conducted six experiments with various speeds of the players (simulating slow, medium, and fast players). The results show that it is possible to evaluate a difficulty curve, for a given game, with a given challenge whose difficulty can be tuned according to a given parameter. However, their proposed method has proven to be difficult to generalize, and it has not been tested with more complex games, having a much bigger parameter space, such as platformers.

Another example is the work of Kreveld et al. [37]. In their work, they proposed a method for automatically rating the difficulty of puzzle game levels using many aspects such as level size, number of features, and so on. They combine all these variables into a difficulty function using a standard linear regression. They also tested their approach with three simple puzzle games: *Flow*,²⁴ *Lazors*,²⁵ and *Move*.²⁶ Their results show a correlation of less than 1, on a scale from 1 to 10, between the computed difficulty level and the one attributed by human players. We found their approach interesting and in line with our proposition. However, they only consider the static element of the level, completely avoiding the dynamic aspect of the game (i.e., moving enemies). Also, their method has not yet been tested with more complex rules.

One of the closest related works we found is the study by Mourato et al. [27]. In their research, they introduced a novel approach to quantifying difficulty in platform games by employing basic principles of probability. The researchers modeled the elements of the level based on the heuristic approach of Reference [11]. They also defined the progression of actions within a level and associated them with failure probabilities to calculate the overall probability of successfully completing the level. Additionally, they incorporated resilience parameters inspired by the work of Reference [14] to capture a user's likelihood of giving up on more challenging tasks. Moreover,

¹⁹<https://mario.nintendo.com/history/>

²⁰<https://www.sega.com/games/sonic-hedgehog>

²¹<http://www.supermeatboy.com/>

²²<https://www.callofduty.com/ca/en/>

²³<https://www.warnerbros.com/games-and-apps/batman-arkham-knight>

²⁴<https://www.blog.bigduckgames.com/>

²⁵<https://www.pyrosphere.net/lazors/>

²⁶<https://www.appbrain.com/app/move-a-brain-shifting-puzzle/com.nitako.move>

they utilized fundamental projectile principles to estimate the difficulty of individual obstacles, representing them as probability values that contribute to the overall success probability calculation. They tested their approach on *Super Mario Bros.* levels, just as we did, and demonstrated that estimating difficulty based on probability provides a satisfying understanding of difficulty progression. However, their model exhibits several limitations. First, all the probabilities in their model are manually crafted without clear justification for their chosen parameters. Second, their model measures the probability of “success” using vague and informal concepts such as player resilience. Last, their model does not account for the dynamic aspect of levels, focusing solely on static objects. This limitation is particularly significant, which prompted us to incorporate the dynamic aspect into our own model.

Other works, such as the work of Yannakakis, Shaker et al. [24], and the work of Constant et al. [12], have proposed measuring the level of difficulty through direct experimentation. Yannakakis’s team introduced the **Platformer Experience Dataset (PED)**,²⁷ which, to the best of our knowledge, is the first open-access game experience corpus [24]. The PED contains multiple modalities of user data from *Super Mario Bros.* players, which is highly relevant to our work, as we also use this game as a test-bench for our approach. The PED was created using a meticulous data collection protocol involving 58 volunteers who participated in recording sessions. These sessions were conducted in Denmark and Greece with participants from various ethnic backgrounds. Over six hours of recording took place, during which participants played a total of 321 games. The dataset is extensive, including demographic information, gaming frequency, weekly gameplay hours, and prior experience with *Super Mario Bros.* Participants played level A and were given three attempts to complete it, followed by a rating questionnaire assessing their engagement, frustration, and challenge levels on a scale from 0 to 4. The process was repeated with a different game level (level B) and another rating questionnaire. After completing both games, participants were asked to indicate if the two were equally engaging, frustrating, challenging, or equally not preferred. In addition to user self-assessment of the difficulty with questionnaires, in their previous work [24, 33], the team also used head movement and other non-verbal physical cues to assess the difficulty. While this dataset, which is quite unique, gives good insights about the perception of challenge by different players in *Super Mario Bros.*, it is very difficult to exploit it in our research. To be usable for comparing our work, we need to access the composition of the used level (for instance, a screenshot of the complete map of each level). However, the researchers do not provide the details about the levels A and B for each player. Moreover, they did not use the original Mario game like us, but a clone named *Infinite Mario*.²⁸ This game provides randomly generated levels for each game, which means that their players have each played different levels. Therefore, we cannot compare directly their difficulty evaluation with our approach, because we cannot compute the difficulty of their game levels with our method (not having the schema of the randomly generated levels). As mentioned in the introduction, these works are *Player-centric*, while our contribution is more *Game-centric*.

Similarly, Constant et al. [12] also investigated player perceptions of difficulty through direct experimentation based on previous research in cognitive psychology. They conducted experiments using a custom game in a museum with 80 participants. The results showed the effectiveness of their method in objectively measuring game difficulty in this specific context. This type of approach is essential, as it allows designers to evaluate game difficulty with a relatively small number of playtesters. While providing valuable insights for optimizing game difficulty, this approach still requires the involvement of numerous players to accurately measure difficulty and is not automated.

²⁷<https://www.ped.institutedigitalgames.com/>

²⁸<https://www.smbgames.be/infinite-mario.php>

Indeed, with the growing popularity of artificial intelligence and ML approaches, recent works, such as the one by Rose et al. [31], proposed measuring difficulty with ML algorithms [35] in platformer games by utilizing player data related to performance, emotion, and other factors. The researchers exploit game telemetry to acquire performance data and the affective state of the player, which is estimated through physiological data obtained from the **Electrodermal Activity (EDA)** of the skin. However, one limitation of such ML-based approaches based on sensors is the need for a fully functional game, with a large player base, and substantial infrastructure of sensors, with a lot of gathered data for it, to be effective. In other words, these models can determine the difficulty of a game level after numerous attempts of the players. Therefore, after extensive training, the model could predict whether a new level within the same game, featuring same elements and mechanics, would be challenging or not. Nevertheless, these approaches are unable to predict the difficulty level of a game during the design process when only a few or no players have experienced it.

In summary, most prior works aimed to measure automatically the difficulty in a game and focused only on the static elements of the game, avoiding dealing with enemies and moving obstacles. Also, most of them are hard to generalize and have been tested only with uncommon games. Several works proposed using DDA models, but these models are not applicable in the early design process of the game and required a lot of data and the involvement of many playtesters. Moreover, it is evident that **SGB (static game balance)** models, which do not incorporate **dynamic difficulty adjustment (DDA)**, are still commonly used in the game industry. Consequently, there is a clear need for new approaches that can quickly measure difficulty without relying on extensive playtesting, particularly to aid in the game design process.

We can see there are two schools of difficulty evaluation approaches that can be applied to two different stages in the development of a video game. During the design and development of a video game, the game designer starts with the game concept and defines the grammar of the game (defining rules, actions, mechanics, etc.). The level designer uses all these basic elements to create levels that will offer different challenges to players. During this process, game and level designers can use approaches that exploit information from the game to propose an estimate of the difficulty level of the proposed challenges. The other school of approaches is positioned downstream of the playtesting stage. Indeed, once a first version of the levels is available, players can try them out, obtaining data that can be processed to estimate difficulty. These analyses can then be fed back to the designers, who can adjust the level accordingly. During these stages, approaches can exploit biometric data obtained from players or generated by game logs. Figure 2 shows a schematic diagram of the development process and the positioning of the different families of approaches within this process. Our proposal falls into the first family of approaches, which is based solely on the game information defined by the designers.

3 Model and Implementation

Our model for evaluating the difficulty in platformer video games is based on the “Move and Avoid” bricks model proposed by Djaouti [15], where the player must reach a certain zone using an avatar (“Move” brick) while avoiding obstacles such as enemies or hazard (“Avoid” brick). Our approach is as follows: First, we extract the collision geometry of a given level. In a clearer context, only objects that potentially produce collision data are considered part of a level, while superfluous objects such as art assets or textures are omitted entirely. Possible extraction methods include using convex hull algorithms such as Graham scan [19], Quickhull [6], or Chan’s algorithm [9], using a computer vision software library such as OpenCV²⁹ [7], or techniques based on machine

²⁹<https://www.opencv.org/>

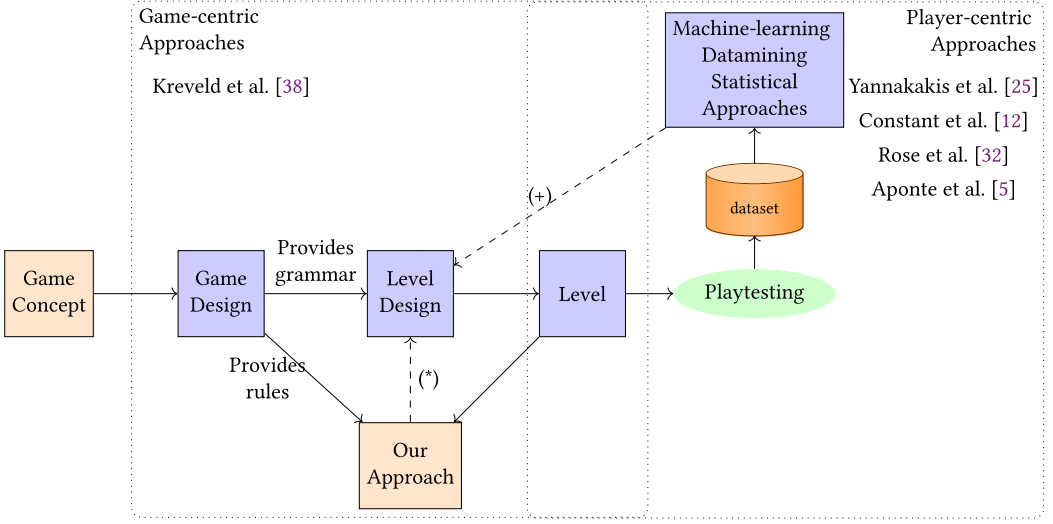


Fig. 2. How we position our approach compared with the state-of-the-art. (*) Game-centric difficulty estimation. (+) Player-centric difficulty estimation.

learning and artificial intelligence [40]. Regardless of the algorithm used for obtaining the collision geometry of a level L , one obtains a 2-partition $B \cup W$ of $\mathbb{Z}^2$, where B is the finite subset of $\mathbb{Z}^2$ composed of the collision pixels; this is called the various obstacles, and W is the subset $W = \mathbb{Z}^2 \setminus B$. Besides, the pixels of the entire level are separated in two distinct sets: Black pixels (i.e., pixels in B) representing pixels interacting with the player avatar such as platforms, enemies, or flames, and White pixels (i.e., pixels in W) that constitute the “empty space” of the level. Significantly, the sets B and W are disjoint, since any pixel either produces a collision or not. For example, the “empty level” is the unique level such that B is the empty set. In addition, in our model, pixels are identified to the discrete plane $\mathbb{Z}^2$ via their Cartesian coordinates. Undoubtedly, 3D games use the space $\mathbb{Z}^3$, but the ideas presented in this article would remain the same.

Depending on the physics of the game as well as the properties of the player avatar, not every white pixel of a level may be reachable by the player. Similarly, some pixels of W may put the player in precarious or harmful situations (e.g., holes, lava, enemies) while others are essential to the player’s progression (e.g., ground, platforms). We further categorize the white pixels of a level into two sets: Reachable and Danger pixels. By extension, any connected reachable (respectively, danger) set of pixels is called a reachable area (respectively, danger area). Note that reachable and danger pixels are not mutually exclusive sets, since most danger areas must also need to be reachable to pose a threat to the player. For example, in *Super Mario Bros.*, the pixels near and above a bottomless pit are reachable and dangerous, while the pixels near and above an empty platform are reachable but not dangerous.

In this article, we use pheromones to abstract the various reachable and danger areas of L . In our model, each object emits two types of pheromones: Type A pheromones represent spaces reachable by the player and type B pheromones represent spaces where some form of danger to the player is present. Formally, for every pixel of an object, we set $(\vec{v}(t), dq(t))$, where $\vec{v}$ is the velocity vector of the pheromones and $dq(t)$ is a function denoting the rate in which pheromones evaporates. For example, the ground only emits type A pheromones with $(\vec{v} = (0, 1), dq(t) = 0)$, effectively creating a permanent reachable space above ground. The case where an object does not emit a certain type of pheromone is easily obtained by setting its respective velocity vector to $\vec{0}$. Bottomless pits are

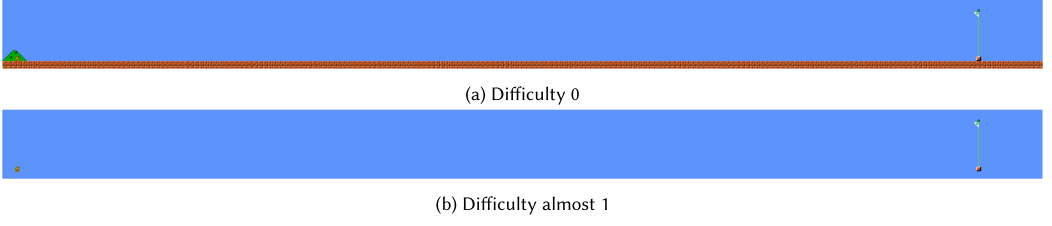


Fig. 3. Two different levels, one with difficulty 0 (impossible to die), and the other one with difficulty 1 (certainty to die).

examples of objects emitting both types of pheromones. Figures 4(b), 4(c), and 4(d) show these spaces for the level 1-1 of the game *Super Mario Bros*.

We applied this modeling of a level to compute the difficulty of *Super Mario Bros*. levels by sliding a window of fixed width over the entire level and computing the total number of pheromones of type *B* for every window. We further refine this process by distinguishing between “static” objects (e.g., holes, spikes) and each enemy type. Thus, we were able to obtain several functions giving the total number of pheromones of a given type at each point in L . We then added those functions to obtain the difficulty function of L , denoted $D(L)$, and integrated the resulting curve over the entire level to compare the relative difficulty of levels. That is, L_1 is “easier” than L_2 if and only if

$$\int_{L_1} D(L_1)dl < \int_{L_2} D(L_2)dl.$$

For example, consider the two levels of Figure 3: L_1 is a level with no type *B* pheromones (e.g., a flat stage with only the ground) (see Figure 3(a)), and L_2 is the “empty level” (i.e., composed entirely of a bottomless pit) ((see Figure 3(b)). It is easy to check that $D(L_1) = 0$ and $D(L_2) = 1$. Furthermore, for any level L , we have

$$D(L_1) \leq D(L) \leq D(L_2).$$

Intuitively, a difficulty of 0 poses no challenge, while a difficulty of 1 is impossible to navigate successfully, no player can complete it, and it is straightforward to extend this modeling to other games. For example, it is certain that the platformer game *Super Meat Boy* has similar mechanics to *Super Mario Bros*., with the notable exception of including a wall jump maneuver. This was handled by our model by simply allowing for walls to emit pheromones, creating new reachable spaces. Further generalization ideas are discussed in Section 6.

The implementation of the model can be seen in Algorithm 1: It uses as data input information of the level’s elements compiled in a JSON³⁰ document, and it includes the position of the elements and their type. Figure 4(a) shows the map that can be constructed with this information. In white are the platforms, in green is the start zone, in blue is the end zone, in brown are the various enemies, and in red are the elements that lead to another part of the level. The first step is to calculate the areas that can be reached by the players (to lay pheromones *A*). For this, we traverse through the level from the departure zone to the end zone and collect all the level’s platform entities. For each platform, we calculate the zones that the avatar can reach. The principle is, the more accessible the areas are (because the player has several ways of reaching them), the more they will have *A* pheromones. A difficult-to-reach area is, for example, one that can only be reached with a jump of maximum amplitude. The platforms can be provided with a preset pheromone *A*. The calculation at this step would therefore be adjusted on the propagation of pheromone *A* according to the other platforms calculated previously. However, in the current implementation, we perform both actions

³⁰<https://www.json.org/>

ALGORITHM 1: Algorithm to compute difficulty metrics.

```

Data: level, windowsize, simulationtime
Result: heatmap, difficultycurve
platformlist ← getPlatforms() from level;
initialize heatmap;
set currentx to 0;
/* First step: add pheromone A to heatmap */
while platformlist is not empty do
    get currentplatform from platformlist;
    add currentplatform's A pheromones to heatmap;    /* Adjust pheromones A according to
        others platforms */
end
/* Second step: add pheromone B to heatmap */
obstaclelist ← getStaticObstacle() from level;
while obstaclelist is not empty do
    /* Part one: add static elements' pheromone B */
    get currentobstacle from obstaclelist;
    add currentobstacle's A pheromones to heatmap;    /* Adjust pheromones B according to
        others platforms */
end
while currentx ≠ levelwidth − windowsize do
    /* Part two: add dynamic elements' pheromone B */
    get currentenemies in currentx + windowsize;
    simulate currentenemies's displacements for simulationtime;    /* Compute all position
        enemies can reach */
    for each enemies' position add B pheromones to heatmap;    /* if an enemies can be at
        several time at a position x,y. The system add pheromone several times at this
        position */
    currentx ← currentx ++;
end
/* Third step: compute difficultycurve from heatmap */
while currentx ≠ levelwidth − windowsize do
    difficultycurve ← add result of 1 on currentx + windowsize;
    currentx ← currentx ++;
end

```

in this step. At the end of this step, we have a heatmap of the level with the A pheromones, indicating all the areas of the level that the character can reach. Figure 4(b) shows the result of this calculation and how we went from the base map (Figure 4(a)) to obtain the map in Figure 4(b) with all pheromones A. White pixels represent pheromones A, and the second step is to complete the level heatmap by calculating the B pheromones (which symbolize danger). We also split this step into two parts; in the first part, we focus on static elements of danger (holes, spikes, and so on—anything that does not involve any movement). We went through the level from the departure zone to the end zone, retrieved each element of danger, and for each element of danger, we were able to retrieve the associated type B pheromone diffusion and adjusted to add it to the heatmap. This adjustment consists in removing regions of pheromone B if the associated threat is blocked by an obstacle. For example, a hole under the ground will not add pheromone B above the ground. Figure 4(c) illustrates this case with the second obstacle. The danger generated by the hole is partially “absorbed”

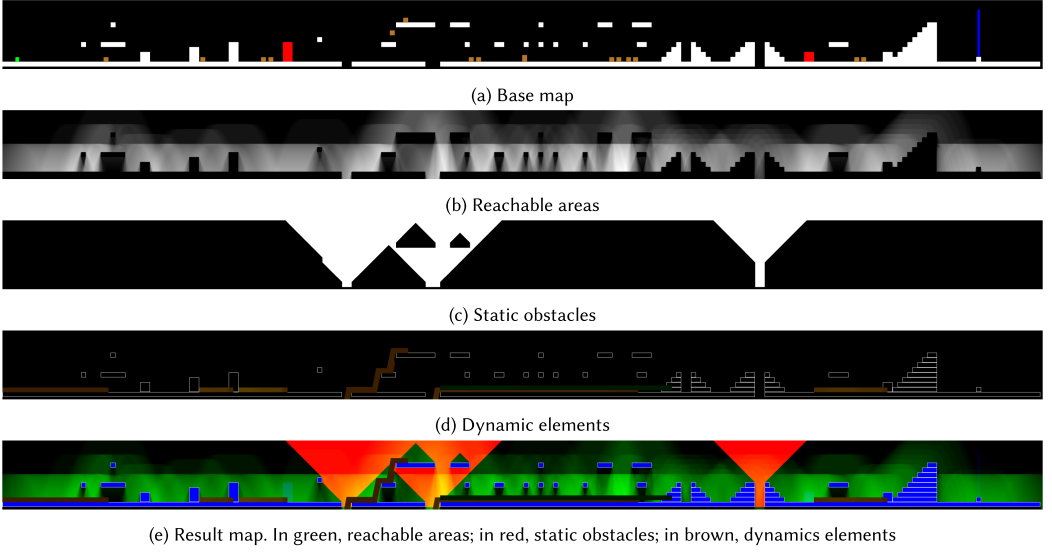


Fig. 4. *Super Mario Bros.* World 1-1 example.

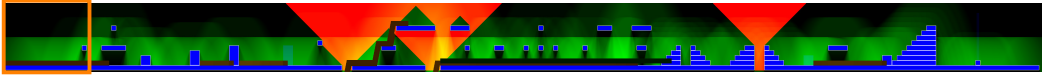


Fig. 5. *Super Mario Bros.* World 1-1 example with window.

by the two platforms above it. In the same figure, the last obstacle has part of its B pheromones “absorbed” by the two pipes to its left and right. Figure 4(c) shows the heatmap with just the static obstacles and in the second part, we focus on the dynamic elements that bring danger. As mentioned in the previous stages, we proceed through the level from the beginning to the end zone. Moreover, we collect the enemies, and for each one, we simulate its movement in the level by running the behavior tree or state machine that defines its behavior to get its position on several images [8]. On top of that, we consider the various obstacles to simulate its movement. For example, in *Super Mario Bros.*, Goombas move in the opposite direction when they encounter an obstacle such as a wall, another enemy, or a pipe. On all of these positions, we added a quantity of pheromone B and calculated the evaporation of pheromone B at the same time. This gives us the pheromone B area covered by the enemy in the level. Figure 4(d) shows the heatmap of pheromone B applied by enemies.

In addition, Figure 4(e) shows the final heatmap of the level with all pheromones A and B at the end of the heatmap calculation process. This completed heatmap is now used to calculate the level’s difficulty curve; it is the final step in the process. To do this, we define a window the height of the level and with a predefined length. Figure 5 illustrates this window for the heatmap calculated above. In this window, we take all the pixels with a pheromone A, and in these pixels, we take all the present pheromones B. Subsequently, we calculated the difficulty of this window by applying Equation (1). This gives us a point on the level’s difficulty curve. In Figure 5, the selected area gives us the first point on the difficulty curve. We move this window by one pixel and repeat the operation to the end of the level to obtain the difficulty curve. Figure 6 shows the difficulty curve for this level.

$$D(L_x) := \frac{|P_B \cap P_A|}{|P_A|}, \quad (1)$$

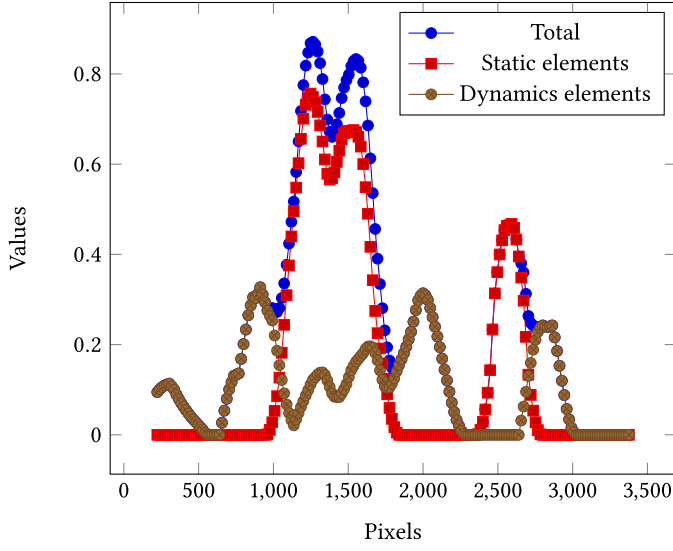


Fig. 6. Level 1-1 data.

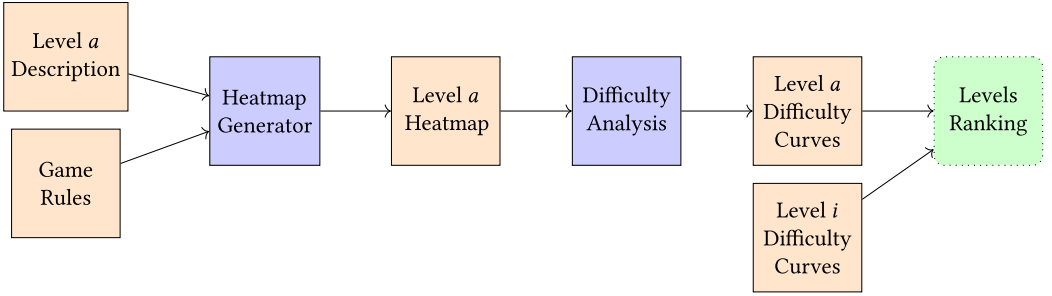


Fig. 7. Overview of our approach.

where L_x denotes the sublevel obtained by taking a window of fixed width such that x is the leftmost horizontal coordinate. By repeatedly shifting L_x by one pixel, we obtain the following curve (Figure 6) where the point (x, y) corresponds to the difficulty of L_x .

Figure 7 summarizes the analysis workflow of our proposal described above. We can see that our proposal inputs the level description (positions of obstacles and enemies, enemy types, topology, etc.) and the game rules (physics rules, possible actions). From these two data, we generate a heatmap of danger zones and accessible zones; we also calculate difficulty curves from this heatmap. And by using several difficulty curves, we can order the levels (or sub-levels) according to their difficulty. The complete code is available on Github.³¹ Keep in mind that in our implementation, pheromones of type A, are emitted exclusively by the player character while simulating its movement throughout the level.

4 First Experiment: Tests, Results, and Discussion

We chose to initially test our approach with the original *Super Mario Bros.* platformer game for the NES console. We selected this game because it supposedly follows the well-known Nishikado

³¹<https://www.github.com/Iannyck/GameChallengeTools>

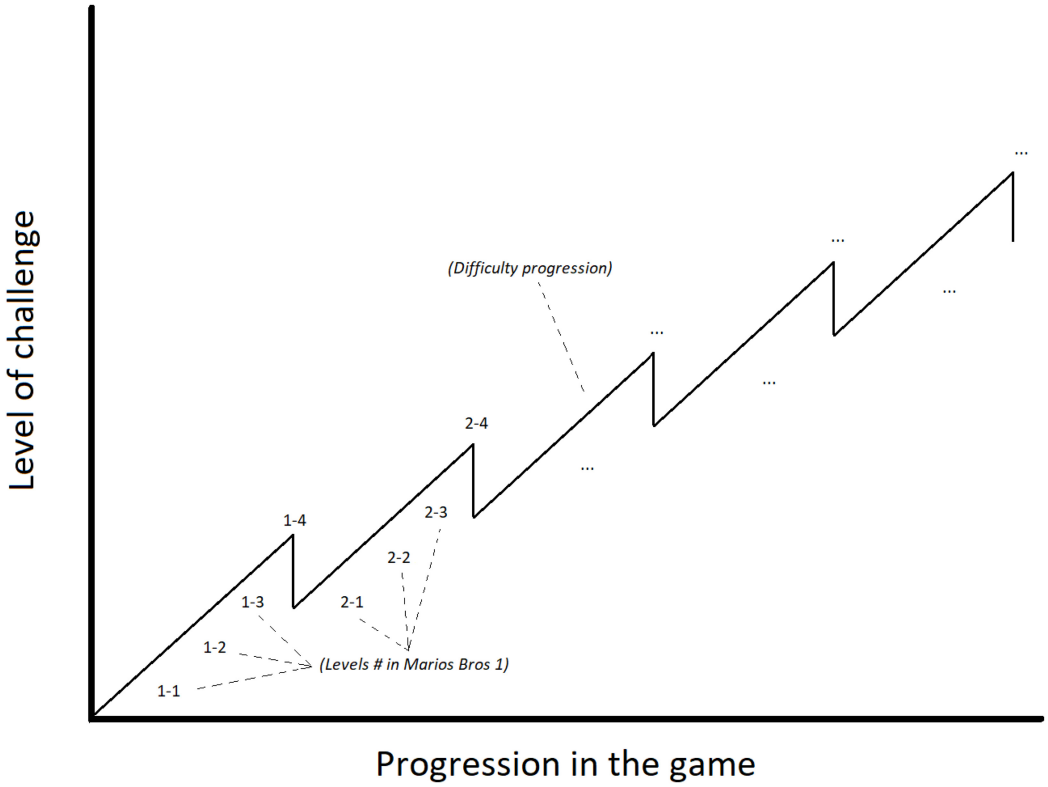


Fig. 8. Nishikado difficulty progression model.

difficulty model [20]. In this model, the difficulty of the levels progressively increases with each level, but at some certain points, it temporarily decreases (see Figure 8) to allow players to master the challenge before ramping it up again. Moreover, this model maintains a linear difficulty progression throughout the game. In *Super Mario Bros.*, the game is divided into different worlds, and each comprises three levels plus a boss level, resulting in a total of 32 levels. In addition, the difficulty intensifies with each level, but it eases after defeating each boss when the player transitions to a new world and explores a different type of environment.

As clearly illustrated in Figure 8, there is a distinct pattern in the difficulty progression. From levels 1-1 through 1-4, the difficulty steadily increases. However, in level 2-1, the design intentionally decreases the level of challenge to be similar to level 1-3, providing a brief respite for players to adapt to the new environment. Subsequently, level 2-2 is designed to match the difficulty of the previous world's boss level, followed by level 2-3, which introduces a new and heightened level of challenge. Considering the well-known design of the game, our initial experiment therefore aimed to validate whether applying our model to *Super Mario Bros.* levels would yield results consistent with the Nishikado curve.

To perform our experiment, we first extracted the position of all elements (platforms and enemies) from the game images. We saved all these elements in JSON files. From these JSON files, we applied our pheromone mapping algorithms and extracted difficulty curves for static and dynamic elements. Figure 6 shows the curves extracted for the first level, for a simulation window 224 px wide, which is the window proposed by the game.

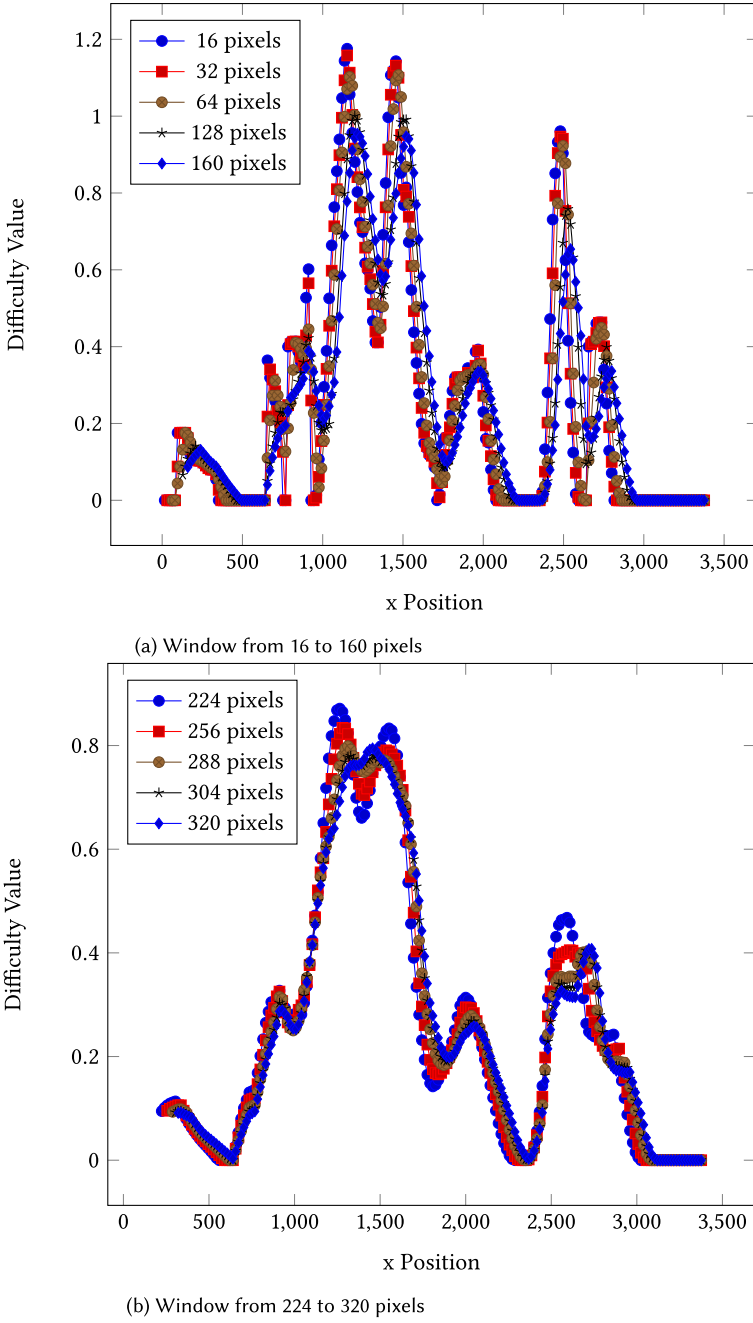


Fig. 9. Difficulty curve for level 1-1 with different windows.

We also experimented with different window sizes to assess their impact on the extracted difficulty curve. Figure 9 shows an example for level 1 of world 1 for 10 different window sizes. We can see that the size of the window has an effect on the maximum value of the curve but that, overall, the curve seems to keep the same shape. The smaller the window, the higher the maximum value,

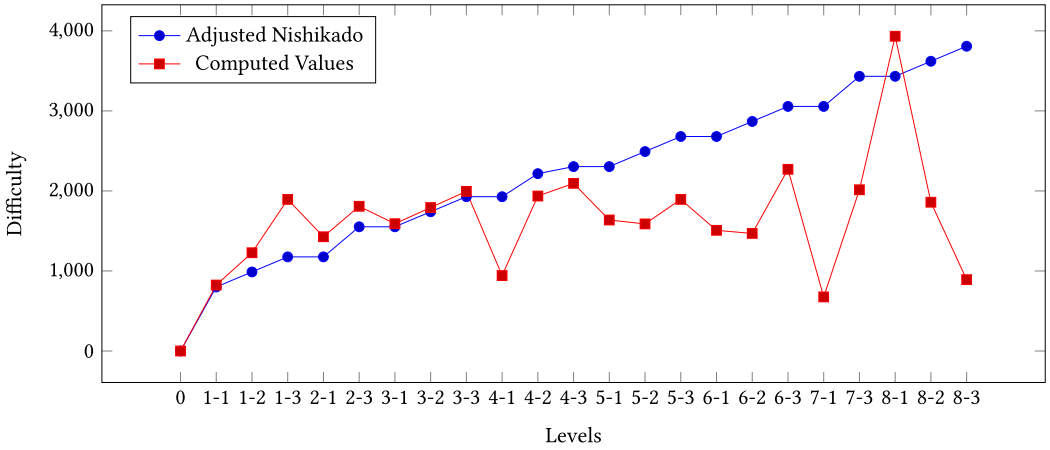


Fig. 10. Adjusted Nishikado and computed difficulty curves.

and the same applies to the slope of the curve. This behavior is logical *a priori*, since the window size is larger, and there are more pixels influencing the difficulty calculation. Furthermore, this behavior is consistent with the impact of a larger window on the game. A larger window offers more information to the players, therefore it is *a priori* easier for them to make a decision.

To integrate the two difficulties into a single curve, we added the two difficulty curves together. However, for enemies, it is important to keep in mind that the amount of pheromone must be adjusted according to the type. Indeed, if we apply the same amount of pheromone to all enemies, then the result will be sensitive to the size of the enemies and their behavior in space, but not to other characteristics such as an enemy's ability to be eliminated or not (by jumping on it in the context of *Super Mario Bros.*, for instance). For example, the impact of enemy characteristics on difficulty can be weighted in our model by the amount of pheromones deposited by enemies.

We applied our approach to calculate the differences in difficulty to the level peers in the order where they are proposed by the game to check whether we were able to obtain a Nishikado progression or not. This calculation gives us Figure 11. At first glance, the difficulty progression of the first world (levels 1-1 to 2-1) is akin to the Nishikado progression, which aimed for a steady progression from level 1-1 to level 1-3, and a decreased difficulty at level 2-1 to meet the approximated difficulty of level 1-3. To formally assess the similarity between the two distributions, we first construct an adjusted Nishikado dataset using the hypothesis that difficulty increases linearly over four levels before dropping to the previous level's difficulty. We use the range [800, 4000] in accordance to our experimental results. Both the adjusted curve and the computed difficulty curves are represented in Figure 10.

Besides, we use a Chi-squared test with 20 degrees of liberty and a 5% significance level to determine whether our computed values correspond to the adjusted Nishikado progression. Without much surprise, the test reveals that both distributions are dissimilar, even though they seem to agree on some segments, namely, for levels 2-1 to 3-1. This is explained by the fact that only common enemies (Goomba, Koopa, Piranha Plant, Buzzy beetle) are considered in the computation. Table 1 shows the difficulty value for these enemies. The remaining enemies, mostly present in the second half of the game, have not yet been modeled, which was explained under our approach limitation part of after world 3. Also, the dynamic objects, such as moving platforms, are actually considered as static objects. Since the difficulty heavily relies on dynamic elements in the second half of the game, we would need to incorporate these enemies to get better results. Indeed, as of

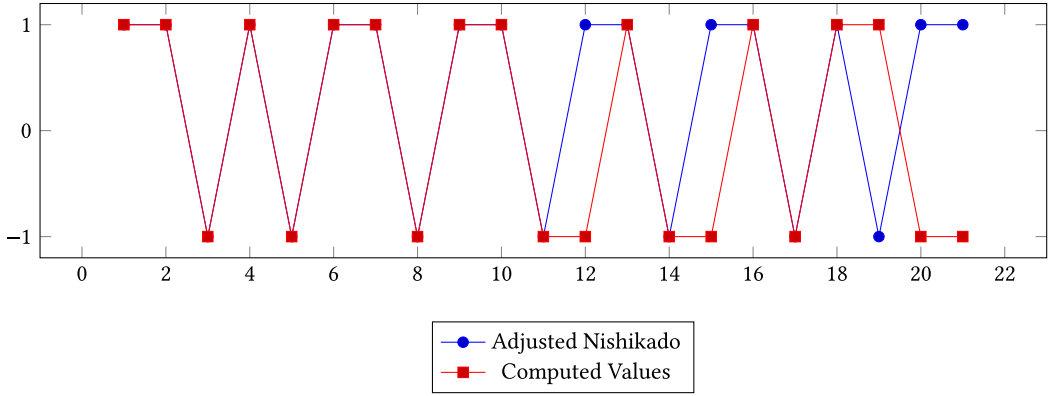


Fig. 11. Comparison of the sign of the derivatives of both the Nishikado and the computed Difficulty curves.

Table 1. Difficulty per Enemy

Enemy Value	Goomba	Koopa	Piranha Plant	Buzzy beetle
	0.110392	0.129857	0.118244	0.126982

present, we did not model very dynamic enemies such as Lakitu, which follows the player through the entire level, or Hammer Bros, which are the only directional enemies that behave according to the player's position. Even though the computed values do not agree with the Nishikado progression, they do agree on the most crucial aspect, knowing that both curves have similar profiles. In other words, the derivatives of both curves have the same sign on corresponding intervals of their respective domains. To test this, we compared the sign of the derivative of each curve, setting +1 for positive derivative and -1 for negative or null derivatives (see Figure 11). Then, a similar Chi square test has been performed. Again, the result falls outside the accepting region, but with a much smaller margin, as both profiles match perfectly on 16 of the 21 intervals. This further proves that our model correctly models the difficulty progression of the first half of the *Super Mario Bros.* game.

It should be noted that, as a *Game-centric approach*, the definition of executive or motor difficulty implemented in our model concerns the geometry of the level and the presence of enemies on the field (with pheromones). As mentioned before, the actual version of the model considers the presence of danger (holes, spikes of any lethal platform) and the four common enemies: Goomba, Koopa, Piranha Plant and Buzzy beetle. Our vision is to provide the most objective computation of motor difficulty based on the geometry, placement, and movement of each of these game elements. However, in the literature, researchers like Yannakakis et al. have argued that difficulty should also be measured in terms of the player's perception [30, 33, 34, 39]. These kinds of works are *Player-centric*. Of course, some players may have specific issues with timing jumps in *Super Mario Bros.*, while others may struggle with fighting enemies. For the former, the perception of difficulty may be higher in a level with a lot of precise jumps and fewer enemies, but for the latter, the difficulty may be perceived as lower due to the limited presence of enemies. Difficulty for a particular individual may vary greatly, depending on their personal skills and capacities, and our goal is to provide a general assessment of difficulty without considering these personal characteristics. However, some of the cited elements are evaluated indirectly. For instance, the difficulty in timing jumps is not measured *per se*, but calculating the number of holes and their size provides an objective general idea of the jumping difficulty. More precision could be added by considering the number of successive holes in the same area. Regarding the enemies, the pheromone mechanic allows a good

general idea of the difficulty in avoiding enemies in a given level. In conclusion, our objective is to provide an practical tool for the designers to evaluate the impact of each choice (adding a hole or removing spike for instance) in creating a level without targeting a specific type of players of skills.

5 Comparative Analysis with Real Players

As observed in our initial experiment, we computed the difficulty of each level of *Super Mario Bros.* using our proposed approach and compared the results with the intended design of the difficulty progression, following the Nishikado curve. This experiment revealed that our approach yields consistent results that correlate with the Nishikado curve. However, it raises the question: What if, in reality, the difficulty of each level in *Super Mario Bros.* does not align with the Nishikado curve as intended by the game designers? What evidence supports the notion that players perceive the difficulty progression as intended?

Our objective in this phase, in fact, was to accurately measure the effective difficulty experienced by players. Naturally, our initial thought was to locate a pre-existing ranking by players available on the internet, given that *Super Mario Bros.* is a renowned classic game that has been around since 1985. Surprisingly, there are not a lot of works that have undertaken the task. The most comprehensive work on the subject seems to be the one completed by Yannakakis, Shaker et al. [24]. They created the **Platformer Experience Dataset (PED)**,³² which contains open-access multiple modalities of user data from players. However, with our approach the use of this dataset is limited as there was no applicable information about the structure of the level found (e.g. enemies, obstacles, goals). Further, the game screenshots used in the article suggest that the game studied is not the original *Super Mario Bros.* but a similar game using procedurally generated levels. Nevertheless, the authors have been provided useful information, namely, the candidates' past gaming experiences, number of deaths, and perceived difficulty of each attempt. However, without any concrete information about the structure of each level, we could not establish a formal ranking of the game's level based on motor difficulty. Another dataset consisted in *Super Mario Bros.* speedrun data, which are easily retrieved from the [speedrun.com](https://www.speedrun.com)³³ online archives and include completion time for each individual level. Again, the use of this sources is not applicable for our study because the variation between different entries is not mathematically significant and it relies heavily on the specific skills of the player.

To alleviate these limitations and enhance the contribution, we decided to conduct an experiment involving real participants. In this experiment, human subjects had to play the levels of *Super Mario Bros.* and rank them according to their perceived level of difficulty. The results of this second experiment provided us with a second means to further validate our model and also make an additional contribution to the community.

5.1 Experimental Protocol

The experimental protocol for this study is as follows: We employed a "Nintendo Classic mini"³⁴ console, featuring 30 pre-loaded games, one of which is *Super Mario Bros.* Figure 12(a) shows our setup for this experiment. This console proved to be invaluable, as it allowed us to create save states, effectively granting us control over the players' experiences. For every level in *Super Mario Bros.*, a dedicated save state was created. This approach provided us with the capability to instantly initiate gameplay at any specific level, eliminating the need to start from the game's beginning. To ensure a consistent and uniform assessment of the difficulty, each save state was carefully designed

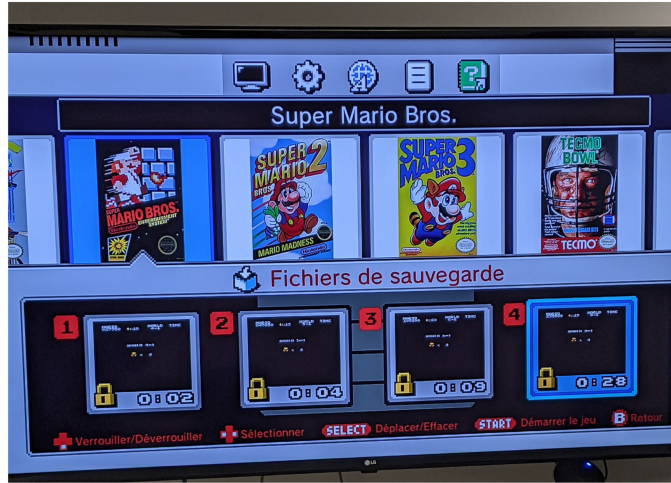
³²<https://www.ped.institutedigitalgames.com/>

³³<https://www.speedrun.com>

³⁴<https://www.nintendo.co.jp/clv/index.html>



(a) Setup



(b) Save states of worlds 4, 5, 6, and 8

Fig. 12. Second experiment setup with “Nintendo Classic mini” console and *Super Mario Bros.* video game.

to place Mario at the start of each level without any power-ups (see details in Figure 12(b)). In the actual game, if a player completes a level with a power-up (e.g., the ability to throw fireballs while Mario is in a taller state), then they would naturally commence the subsequent level with the same advantage. Consequently, if a player began a level with such a power-up, then it would inevitably skew their perception of the difficulty, making the level appear easier compared to a player who starts without any power-ups. Hence, the use of save states enabled us to have precise control over the levels the players engaged with, ensuring that all participants started each level on an equal footing without any varying advantages.

For the experiment, we recruited 26 candidates via an e-mail announcement in the **University of Québec at Chicoutimi (UQAC)**³⁵ employees and students mailing lists. Any person 18 years

³⁵<https://www.uqac.ca>

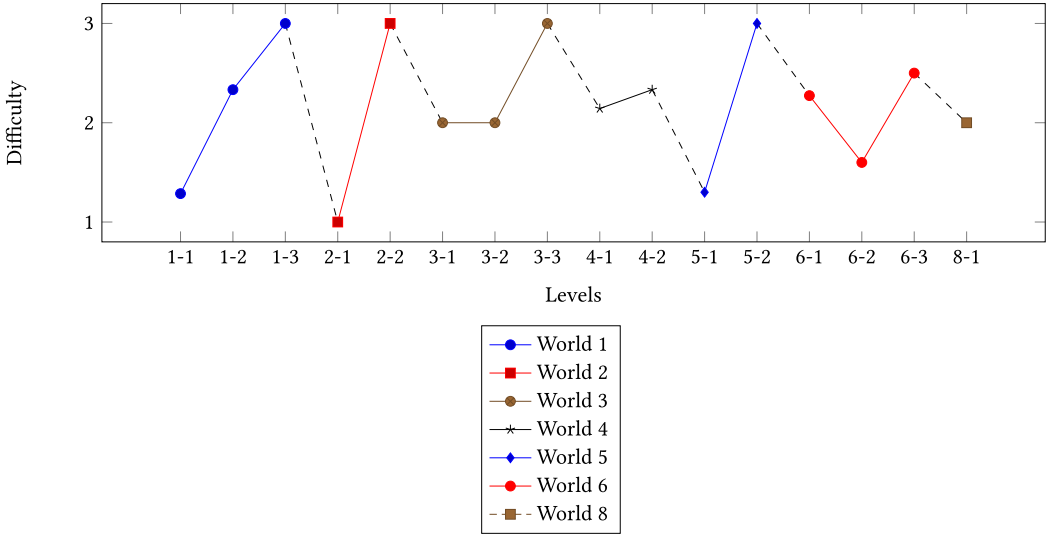


Fig. 13. Mean perceived difficulty of levels played by the candidates. The dashed line denotes the changing of world.

old and above was eligible to participate. To increase the variety of player profiles, we recruit the candidates despite of their gaming background as well as cognitive or physical limitations. After a candidate manifested the interest in participating, we organized a 15-minute individual play session. First, the participants had a chance to familiarize themselves with the game by playing without constraints from the beginning of level 1-1 for as long as needed. This introductory part is not included in the results. After the candidate felt acquainted enough with the game, they played a randomized selection of three levels. For each one, the candidate had at least three lives. We recorded the time needed to finish the level or get a game-over screen and the number of deaths. Also, after successfully completing a level, the player was allowed to continue on to the next level. Note that water levels such as 2-2 were excluded due to their particular physics mechanics. At the end of the session, the participants had to complete a short survey where we asked for their prior experience with video games and *Super Mario Bros.*, their favorite types of games, and their own assessment of their ability towards platformer games, an estimate of the time it took to complete each level, the number deaths, and a ranking of the three played levels in order of perceived difficulty. The results are presented and analyzed in the next section. Finally, it is crucial to say that this experiment was conducted with the approval of our ethical committee under certificate number 2022-882.

5.2 Results and Comparative Analysis

Figures 13 and 14 present the players' ranking (perception) of levels based on their perceived difficulty. Since each participant played only a few levels randomly, for example, 3-1, 3-2, and 3-3, the ranking of difficulty shown in Figures 13 and 14 does not reveal significant differences between different worlds but does show variations between levels within the same world. This is because the players did not have a reference point between worlds for a comprehensive comparison. As a result, we cannot observe a progressive difficulty level throughout the entire game in that figure. To address this, we computed Figure 13, where we calculated the mean perceived difficulty for each level, adjusted by world, based on the players' perceptions gathered during the experiment.

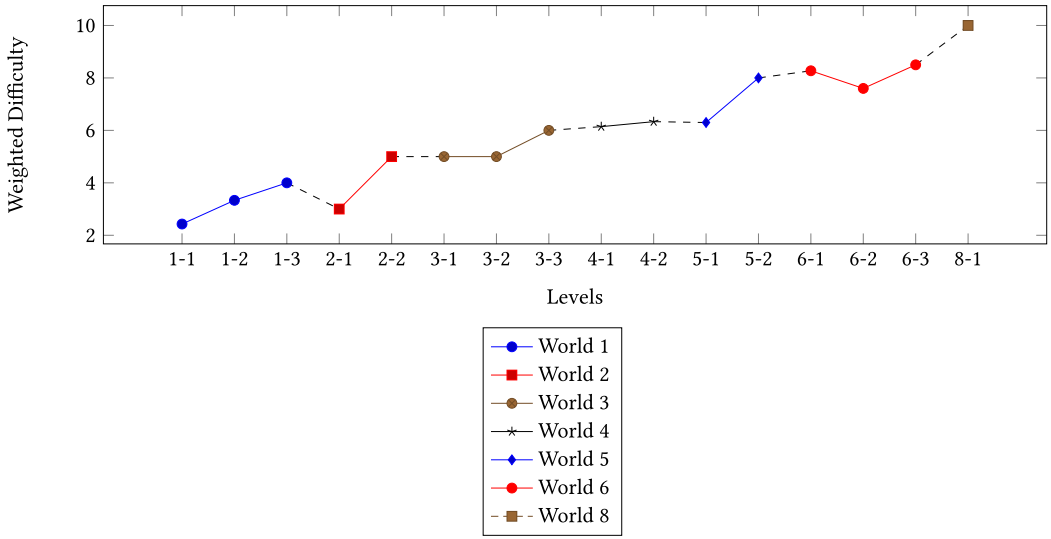


Fig. 14. Weighted mean perceived difficulty of levels played by the candidates. The dashed line denotes the changing of world.

To do this, we simply added +1 to each world in the participant's order starting from world 2. This gave us Figure 14, which indicates that the difficulty level decreases with each change of world, resembling Nishikado's difficulty progression. This pattern closely correlates with the intended game design, suggesting that players perceived the game's difficulty mostly as intended by the designer.

Comparing the results obtained from our automatic model, as shown in the previous section (Figure 11), with the difficulty curve extracted from this second experiment involving real players, we can observe a promising correlation, particularly for the initial stages of the game. It is worth keeping in mind that our current computations only consider common enemies (Goomba, Koopa, Piranha Plant, Buzzy beetle) and do not account for exotic enemies primarily found in the latter half of the game. Additionally, the absence of dynamic objects, such as moving platforms, which are more prevalent in the second half of the game, has an impact on the results. We are confident that including these elements in our model in the future will enable us to calculate a coherent and correlated difficulty progression throughout the game.

Finally, demographic information is presented in Figures 15, 16, and 17. The data show that the participants in the experiment consisted of both occasional and hardcore gamers who played video games several times a week. There were no participants with no prior experience in video games. In terms of their reported skills in platform games, the participant population exhibited a range of skills from average to good. In regards to experience with *Super Mario Bros.*, over two-thirds of the candidates had played the game, with some having completed it multiple times. While this experiment allowed us to establish a player-perceived ranking of levels for comparison, conducting a more extensive investigation in the future with a larger number of participants will help us further refine these results.

6 Generalization

This section presents ongoing and future work to increase the generalization of the approach, including the consideration of more dynamic elements and application to other game genres. For

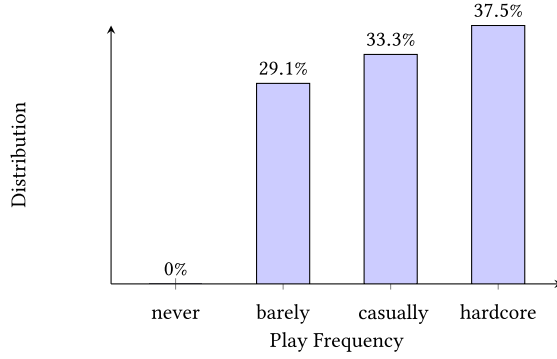


Fig. 15. Playing frequency.



Fig. 16. Self-assessment about skills in platform games.

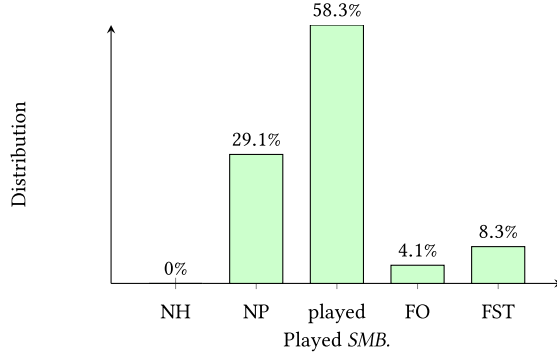
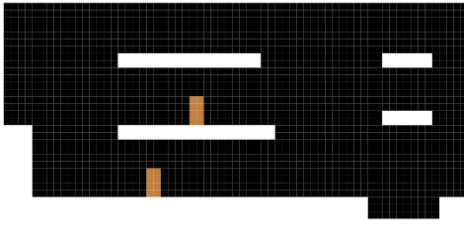
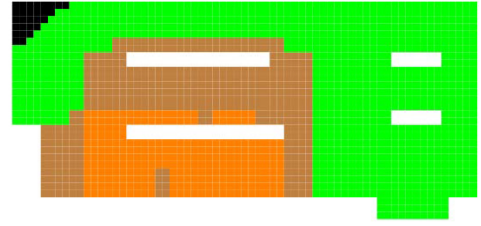


Fig. 17. Have you played *Super Mario Bros.* before? NH: never heard; NP: never played; FO: finished once; FST: finished several times.

dynamic elements, our approach is basically to simulate the element's behavior (based on the state machine or behavior tree that defines its behavior). The approach that was applied to enemies can also be applied to mobile platforms. The difference is that pheromones A (reachable points) must be deposited for each possible platform position. Some enemies are more complex to simulate, as they take the player's actions into account; for example, *Hammer Bros.*, and Lakitu-launched projectiles that target player's avatar. For the *Hammer Bros.*, a half-octagonal zone (because we



(a) In brown, the position of Hammer Bros.



(b) In brown and orange, the pheromones deposited by the Hammer Bros. Orange indicates a greater quantity of pheromones.

Fig. 18. Extract from Super Mario Bros. level 3-1, in which the Hammer Bros. appear for the first time.

could cut it off at the ground line) can be defined around the enemy, in which the player is in danger of being hit by the hammers thrown by the enemy. Throughout this zone, we can deposit the pre-defined quantity of pheromone B. Figure 18 shows an example of this type of enemy. For a Lakitu, the enemy itself deposits a predefined amount of pheromone B on its line of movement, but also on the entire level, because as long as it is alive, it can potentially throw Spiny's eggs all over the level. This approach can be refined by removing the areas covered by platforms that Spiny's eggs cannot traverse.

First, other 2D platformers are prime candidates for applying our model. For instance, *Super Meat Boy* has similar mechanics to *Super Mario Bros.*, with the added possibility of wall jumping, which is handled by allowing walls to emit green pheromones. Another example is the 2D *Mega Man* games, which are essentially platformer games with an added shooting mechanic; this has the effect of reducing the red pheromones emitted by the enemies.

Second, the model presented in Section 3 is directly applicable to other 2D games following the move-avoid paradigm, such as shooter-type games (e.g., *Gradius*³⁶) and stealth-type games (e.g., *Metal Gear*³⁷). Note that the sliding window method for constructing difficulty curves is generally not applicable to the latter, as these levels are typically constructed on a map allowing for any 2D movement, eschewing the left-right design of platformers and shooters. This could be remedied by considering a preferred path for the player and evaluate the quantity of pheromones at pre-determined discrete steps along the path.

It is plain sailing to generalize our mathematical model to the three-dimensional case by simply considering sets of voxels of the space $\mathbb{Z}^3$. Needless to say, such a model is hardly applicable in practice, since it would require excessive computing power.

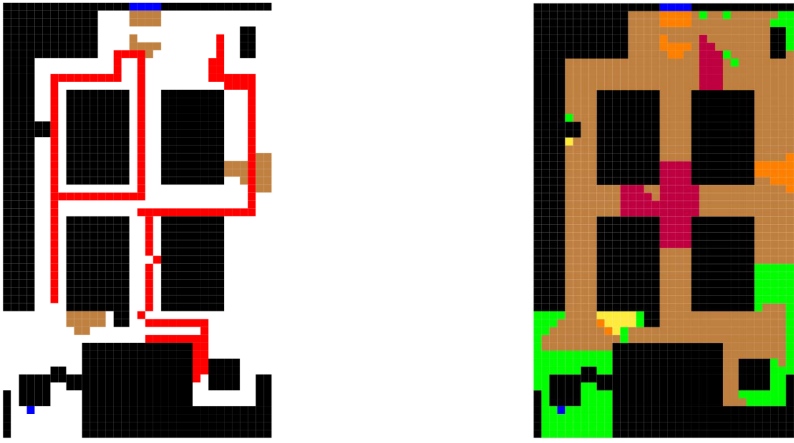
Nevertheless, our model is directly applicable to certain 3D games such as *Metal Gear Solid*,³⁸ *Tenchu*,³⁹ and so on, since it essentially uses a 2D gameplay approach and a top-down view. In this particular example, reachable zones consist of the space the player can move into, and danger zones are obtained from the guards' paths (see Figure 19). For 3D games that are not based on pixels, we propose to use Navigation Meshes to define reachable areas [38] and an adjacency graph structure to encode pheromones. Those cases require slightly more attention in computing the difficulty metrics under Section 3, since it is complicated to define a left-to-right sliding window. Again, one could instead define a preferred path for the player and evaluate the quantity of pheromones at pre-determined discrete steps along the path.

³⁶https://www.konami.com/games/us/en/products/dl_gradius_tb_e/

³⁷<https://www.konami.com/mg/mc/>

³⁸<https://www.konami.com/mg/archive/mgs/>

³⁹<https://www.tenchu.net/>



(a) Patrol path of the two level guards. The patrol path is visible in red. Elements that emit noise (e.g., puddles and metal plates in this level) and can alert guards when the player's avatar steps on them are in brown.

(b) Pheromones deposited by guards and the player's possible displacements in the level. Green indicates reachable areas. Yellow indicates a low level of danger pheromone. Brown indicates a medium level of danger pheromone. Orange indicates a medium-high level of danger pheromone. Purple a high level of danger pheromone.

Fig. 19. Example of our approach on a level similar to the “Cargo Dock” (first level) of *Metal Gear Solid*, an action and stealth video game. In blue, bottom left, the player's starting point. In blue at the top, the level's end zone. The third guard, who appears when the player reaches the end of the level via the end zone, is not considered in this example.

7 Conclusion and Future Works

Successful games rely on a well-balanced difficulty curve, which is challenging to design [26]. The process of developing a game is complex and time-consuming, requiring substantial input from playtests. Moreover playtesters are hard to find, and their input is indispensable. In addition, designers need to be able to objectively assess, at least roughly, the difficulty of a game's levels to balance it before requiring playtesters. Furthermore, a promising approach to address this challenge involves leveraging software tools capable of automatically assessing the difficulty of game levels, thereby reducing the reliance on extensive playtesting [29]. This kind of solution can be used in the early stages of the design process, when game parameters and mechanics often change.

In this article, we propose a novel formal model designed to assist game designers in the creation of platformer-type games. Our model facilitates the automatic measurement of executive or motor difficulty within a level by analyzing its statics and interactive elements. By identifying potential areas of static danger (such as holes and spikes) and by simulating dynamic dangers through enemy movements using a pheromone algorithm [17], our model assesses the motor skills difficulty of a specific level, effectively eliminating the need for exhaustive playtesting. We first formalized our proposal, showed the algorithms, and subsequently presented its implementation. We then presented the results of a complete experiment conducted on all the levels of *Super Mario Bros*. The promising results of this first experiment showed the effectiveness of our automatic algorithmic assessment model in providing highly coherent results that align with the theoretical difficulty curve for this game, which aimed to follow the Nishikado difficulty curve [20]. In addition, we provided a comparative analysis of our results with a study that we conducted directly with playtesters. Our model offers a high level of generalizability, as it can be easily applied to other types of games, such as platformers, shooters, and stealth-type games. For instance, our model is directly applicable to

the *Super Meat Boy* and *Mega Man* series, since the games' obstacles are more or less identical to those in *Super Mario Bros.* (e.g., *Mega Man*'s arm cannon and *Super Mario Bros.* fire flowers have similar properties).

As a matter of course, it is important to acknowledge the limitations of our current work, which we aim to address in future research. The first to be mentioned is the actual version of our model that separately measures the static and dynamic difficulty at the same time, without normalizing both curves, or computing a single combined measurement for the overall executive difficulty. While adding both curves together may seem like an obvious solution, we are committed to developing a more coherent normalization approach. One possible approach is to analyze the mathematical properties of the curves, potentially yielding interesting results. For instance, the derivative gives a formal analysis of the difficulty's rate of change throughout each level (and, by extension, the whole game), which in turn provides useful information to the level designer. Another existing limitation is the dynamic assessment of difficulty. Although the pheromone algorithmic approach shows promise, we recognize the need to consider additional types of enemies and dynamic elements. Currently, our model assigns, for instance, the same amount of pheromone to a regular Buzzy beetle as it does to a Buzzy beetle with spikes. However, in reality, not all enemies pose equal danger, and we do account for the size of enemies, where larger enemies generate more pheromone. Nevertheless, we acknowledge the necessity of implementing a formal categorization system for different enemy types, which would assign varying amounts of pheromone based on their relative danger and size.

In our future works, we plan to expand the testing of our model to other platformer games such as *Mega Man*. We also want to reduce the limitations of the enemies' type as mentioned in the previous paragraph. Last, we aim to develop an extension of our model, which will consider the two other types of difficulty (comprehensive and strategic), and we believe this addition would enable the model to be applied to different kinds of games beyond platformers.

Appendix

A Research Methods

A.1 Survey Form

The following are the questions submitted to the candidates (translated from the original language into English).

- Do you play video games?
 - Never
 - Barely (less than once per week)
 - Occasionally (at least 1-3 times a week)
 - Frequently (almost everyday)
- What are your favourite types of games?
 - None (I do not play video games)
 - Platformer (Ex: Super Mario)
 - Shooter (Ex: Halo, Uncharted)
 - Racing (Ex: Forza, Gran Turismo, Mario Kart)
 - Strategy (Ex: Civilization, Starcraft)
 - Other (give more precisions)
- Have you played Super Mario Bros.?
 - Never and I do not know this game
 - Never and I do know this game
 - Yes but I have never completed it

- Yes and I have completed it
- Yes and I have completed it several time
- How do you rate your skills in platformer games?
 - Very low (I think I will have trouble finishing most of the levels)
 - Medium (I think I can finish some levels)
 - Good (I think I can finish most levels)
 - Very good (I think I can finish all the levels)
- For the first level played
 - How many times have you tried it?
 - Have you completed the level?
 - No
 - Yes:(if yes). How long did it take you to complete it?
- For the second level played
 - How many times have you tried it?
 - Have you completed the level?
 - No
 - Yes:(if yes). How long did it take you to complete it?
- For the third level played
 - How many times have you tried it?
 - Have you completed the level?
 - No
 - Yes:(if yes). How long did it take you to complete it?
- Order the three levels according to perceived difficulty (1 being the easiest).

Acknowledgments

We would like to show our gratitude to all the students who contributed to this work, in particular, Simon Lescieux, Mathis Ronzon, Jules Linard, Noah Plourde-Marcoux, and Maxime Morin.

References

- [1] J. T. Alexander, J. Sear, and A. Oikonomou. 2013. An investigation of the effects of game difficulty on player enjoyment. *Entert. Comput.* 4, 1 (2013), 53–62.
- [2] A. Anditya and A. Sihabuddin. 2016. An optimal input for role-playing game’s combat pace using an active time battle system algorithm. In *2nd International Conference on Science and Technology-Computer (ICST’16)*. IEEE, 173–180.
- [3] Maria-Virginia Aponte, Guillaume Levieux, and Stéphane Natkin. 2009. Scaling the level of difficulty in single player video games. In *8th International Conference on Entertainment Computing (ICEC’09)*. Springer-Verlag, Berlin, 24–35. DOI: https://doi.org/10.1007/978-3-642-04052-8_3
- [4] Maria-Virginia Aponte, Guillaume Levieux, and Stéphane Natkin. 2011. Difficulty in videogames: An experimental validation of a formal definition. In *8th International Conference on Advances in Computer Entertainment Technology (ACE’11)*. Association for Computing Machinery, New York, NY, USA, Article 49, 8 pages. DOI: <https://doi.org/10.1145/2071423.2071484>
- [5] S. Natkin, M.-V. Aponte, and G. Levieux. 2011. Measuring the level of difficulty in single player video games. *Entert. Comput.* 2 (2011), 205–213.
- [6] C. Bradford Barber, David P. Dobkin, and Hannu Huhdanpaa. 1996. The quickhull algorithm for convex hulls. *ACM Trans. Math. Softw.* 22, 4 (Dec. 1996), 469–483. DOI: <https://doi.org/10.1145/235815.235821>
- [7] G. Bradski. 2008. Learning OpenCV: Computer vision with the OpenCV library. *O’REILLY* 2 (2008). <https://www.oreilly.com/library/view/learningopencv/9780596516130/>
- [8] Mat Buckland. 2005. *Programming Game AI by Example*. Jones & Bartlett Learning.
- [9] T. M. Chan. 1996. Optimal output-sensitive convex hull algorithms in two and three dimensions. *Discr. Computat. Geom.* 16 (1996), 361–368.
- [10] Jenova Chen. 2007. Flow in games (and everything else). *Commun. ACM* 50, 4 (Apr. 2007), 31–34. DOI: <https://doi.org/10.1145/1232743.1232769>

- [11] Kate Compton and Michael Mateas. 2021. Procedural level design for platform games. *Proc. AAAI Conf. Artif. Intell. Interact. Digit. Entert.* 2, 1 (Sep. 2021), 109–111. DOI : <https://doi.org/10.1609/aiide.v2i1.18755>
- [12] Thomas Constant, Guillaume Levieux, Axel Buendia, and Stéphane Natkin. 2017. From objective to subjective difficulty evaluation in video games. In *Conference on Human–Computer Interaction (INTERACT’17)*, Regina Bernhaupt, Girish Dalvi, Anirudha Joshi, Devanuj K. Balkrishnan, Jacki O’Neill, and Marco Winckler (Eds.). Springer International Publishing, Cham, 107–127.
- [13] Alena Denisova, Christian Guckelsberger, and David Zendle. 2017. Challenge in digital games: Towards developing a measurement tool. In *CHI Conference Extended Abstracts on Human Factors in Computing Systems (CHI EA’17)*. 2511–2519.
- [14] Heather Desurvire, Martin Caplan, and Jozsef A. Toth. 2004. Using heuristics to evaluate the playability of games. In *CHI Conference Extended Abstracts on Human Factors in Computing Systems (CHI EA’04)*. Association for Computing Machinery, New York, NY, USA, 1509–1512. DOI : <https://doi.org/10.1145/985921.986102>
- [15] Damien Djaouti, Julian Alvarez, Jean-Pierre Jessel, Gilles Methel, and Pierre Molinier. 2008. A gameplay definition through videogame classification. *Int. J. Comput. Games Technol.* 2008 (2008).
- [16] Kaleb Eberhart. 2016. Development of difficulty in games. *GameDeveloper.com* (2016), 1–5. <https://www.gamedeveloper.com/design/developmentof-difficulty-in-games>
- [17] Bogdan-Florin Florea, Ovidiu Grigore, and Mihai Datcu. 2015. Pheromone averaging exploration algorithm. In *International Conference on Advanced Robotics (ICAR’15)*. 617–622. DOI : <https://doi.org/10.1109/ICAR.2015.7251520>
- [18] Rhys Frampton. 2021. A new taxonomy of difficulty: A call to reconsider how we talk about difficulty in games. *GameDeveloper.com* (2021), 1–7. <https://www.gamedeveloper.com/game-platforms/a-new-taxonomy-of-difficulty>
- [19] R. L. Graham. 1972. An efficient algorithm for determining the convex hull of a finite planar set. *Inform. Process. Lett.* 1, 4 (1972), 132–133. DOI : [https://doi.org/10.1016/0020-0190\(72\)90045-2](https://doi.org/10.1016/0020-0190(72)90045-2)
- [20] Patrick Holleman. 2018. *Reverse Design: Super Mario World*. CRC Press.
- [21] Ayesha Irfan, Adeel Zafar, and Shahbaz Hassan. 2019. Evolving levels for general games using deep convolutional generative adversarial networks. In *11th Computer Science and Electronic Engineering Conference (CEECE’19)*. 96–101. DOI : <https://doi.org/10.1109/CEECE47804.2019.8974332>
- [22] Beakcheol Jang, Myeonghwi Kim, Gaspard Harerimana, and Jong Wook Kim. 2019. Q-learning algorithms: A comprehensive classification and applications. *IEEE Access* 7 (2019), 133653–133667. DOI : <https://doi.org/10.1109/ACCESS.2019.2941229>
- [23] Tremblay Jonathan, Bouchard Bruno, and Bouzouane Abdenour. 2012. Understanding and implementing adaptive difficulty adjustment in video games. In *Algorithmic and Architectural Gaming Design: Implementation and Development*. IGI Global, 82–106.
- [24] Kostas Karpouzis, Georgios N. Yannakakis, Noor Shaker, and Stylianos Asteriadis. 2015. The platformer experience dataset. In *International Conference on Affective Computing and Intelligent Interaction (ACII’15)*. 712–718. DOI : <https://doi.org/10.1109/ACII.2015.7344647>
- [25] Johann Knorr and Carlos Vaz de Carvalho. 2021. Using dynamic difficulty adjustment to improve the experience and train FPS gamers. In *9th International Conference on Technological Ecosystems for Enhancing Multiculturality (TEEM’21)*. Association for Computing Machinery, New York, NY, USA, 195–200. DOI : <https://doi.org/10.1145/3486011.3486447>
- [26] Jeppe Theiss Kristensen, Arturo Valdivia, and Paolo Burelli. 2021. Statistical modelling of level difficulty in puzzle games. In *IEEE Conference on Games (CoG’21)*. IEEE Press, 1–8. DOI : <https://doi.org/10.1109/CoG52621.2021.9619050>
- [27] Fausto Mourato and Manuel Próspero dos Santos. 2010. Measuring difficulty in platform videogames. In *4ª Conferência Nacional Interação humano-computador*.
- [28] Vince Palban. 2021. Managing difficulty in games. *Game Des. Devel.* (2021), 1–4. <https://ecampusontario.pressbooks.pub/gamedesigndevelopmenttextbook/chapter/managing-difficulty-in-games/>
- [29] Panagiotis D. Paraschos and Dimitrios E. Koulouriotis. 2023. Game difficulty adaptation and experience personalization: A literature review. *Int. J. Hum.-comput. Interact.* 39, 1 (2023), 1–22. DOI : <https://doi.org/10.1080/10447318.2021.2020008>
- [30] Kosmas Pinitas, David Renaudie, Mike Thomsen, Matthew Barthet, Konstantinos Makantasis, Antonios Liapis, and Georgios N. Yannakakis. 2023. Predicting player engagement in Tom Clancy’s The Division 2: A multimodal approach via pixels and gamepad actions. In *25th International Conference on Multimodal Interaction (ICMI’23)*. Association for Computing Machinery, New York, NY, USA, 488–497. DOI : <https://doi.org/10.1145/3577190.3614203>
- [31] Marcos P. C. Rosa, Eduardo A. dos Santos, Iago L. R. de Moraes, Tiago B. P. e Silva, Mauricio M. Sarmet, Carla D. Castanho, and Ricardo P. Jacobi. 2021. Dynamic difficulty adjustment using performance and affective data in a platform game. In *HCI International 2021 - Late Breaking Papers: Design and User Experience*, Constantine Stephanidis, Marcelo M. Soares, Elizabeth Rosenzweig, Aaron Marcus, Sakae Yamamoto, Hirohiko Mori, Pei-Luen Patrick Rau, Gabriele Meiselwitz, Xiaowen Fang, and Abbas Moallem (Eds.). Springer International Publishing, Cham, 367–386.

- [32] W. DeCamp and R. Sevin. 2020. Video game genres and advancing quantitative video game research with the genre diversity score. *Comput. Games J.* 9, 4 (2020), 401–420.
- [33] Noor Shaker, Stylianos Asteriadis, Georgios N. Yannakakis, and Kostas Karpouzis. 2013. Fusing visual and behavioral cues for modeling user experience in games. *IEEE Trans. Cybern.* 43, 6 (2013), 1519–1531. DOI : <https://doi.org/10.1109/TCYB.2013.2271738>
- [34] Tianye Shu, Jialin Liu, and Georgios N. Yannakakis. 2021. Experience-driven PCG via reinforcement learning: A Super Mario Bros study. (2021), 1–9.
- [35] Ali Sohail, Naeem A. Nawaz, Asghar Ali Shah, Saim Rasheed, Sheeba Ilyas, and Muhammad Khurram Ehsan. 2022. A systematic literature review on machine learning and deep learning methods for semantic segmentation. *IEEE Access* 10 (2022), 134557–134570. DOI : <https://doi.org/10.1109/ACCESS.2022.3230983>
- [36] Jonathan Tremblay, Bruno Bouchard, and Abdenour Bouzouane. 2010. Adaptive game mechanics for learning purposes—Making serious games playable and fun. In *International Conference on Computer Supported Education*.
- [37] Marc van Kreveld, Maarten Löffler, and Paul Mutser. 2015. Automated puzzle difficulty estimation. In *IEEE Conference on Computational Intelligence and Games (CIG'15)*. 415–422. DOI : <https://doi.org/10.1109/CIG.2015.7317913>
- [38] Wouter Van Toll, Roy Triesscheijn, Marcelo Kallmann, Ramon Oliva, Nuria Pelechano, Julien Pettré, and Roland Geraerts. 2016. A comparative study of navigation meshes. In *9th International Conference on Motion in Games*. 91–100.
- [39] Ziqi Wang, Jialin Liu, and Georgios N. Yannakakis. 2022. The fun facets of Mario: Multifaceted experience-driven PCG via reinforcement learning. In *17th International Conference on the Foundations of Digital Games (FDG'22)*. Association for Computing Machinery, New York, NY, USA, Article 44, 8 pages. DOI : <https://doi.org/10.1145/3555858.3563282>
- [40] Kai Xu, Vladimir Kim, Qixing Huang, Niloy Mitra, and Evangelos Kalogerakis. 2016. Data-driven shape analysis and processing. In *SIGGRAPH ASIA 2016 Courses*. 1–38. DOI : <https://doi.org/10.1145/2988458.2988473>

Received 22 December 2023; revised 14 July 2024; accepted 29 July 2024